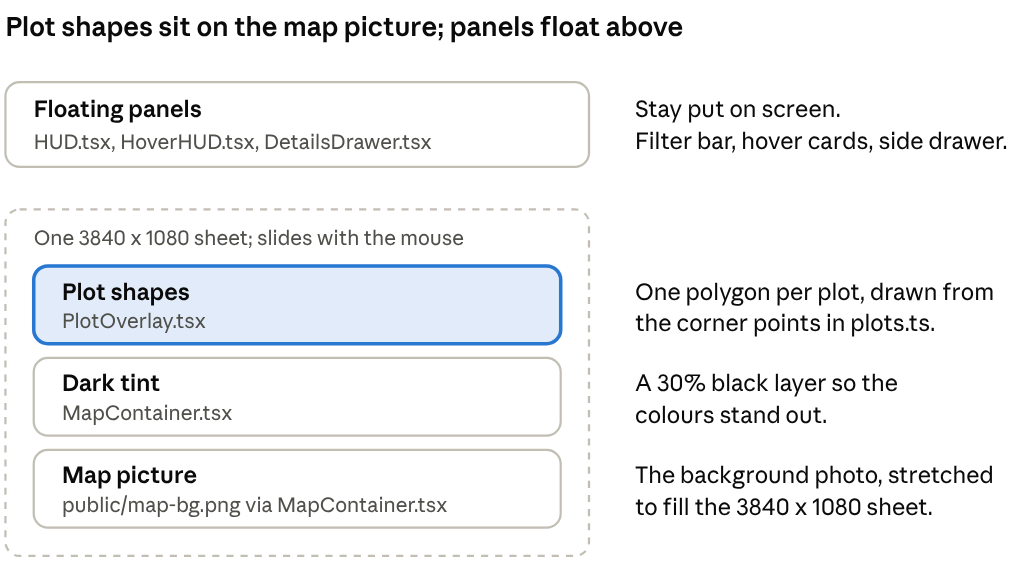
Plot System: How the Polygons Are Drawn

Oct 27, 2026 · @vatsal varma

Every plot on the map is just a list of corner points that the code joins into a coloured shape. To move a corner, you edit its two numbers in `src/data/plots.ts`; this project has no mouse-dragging of points.

The map is a stack of layers

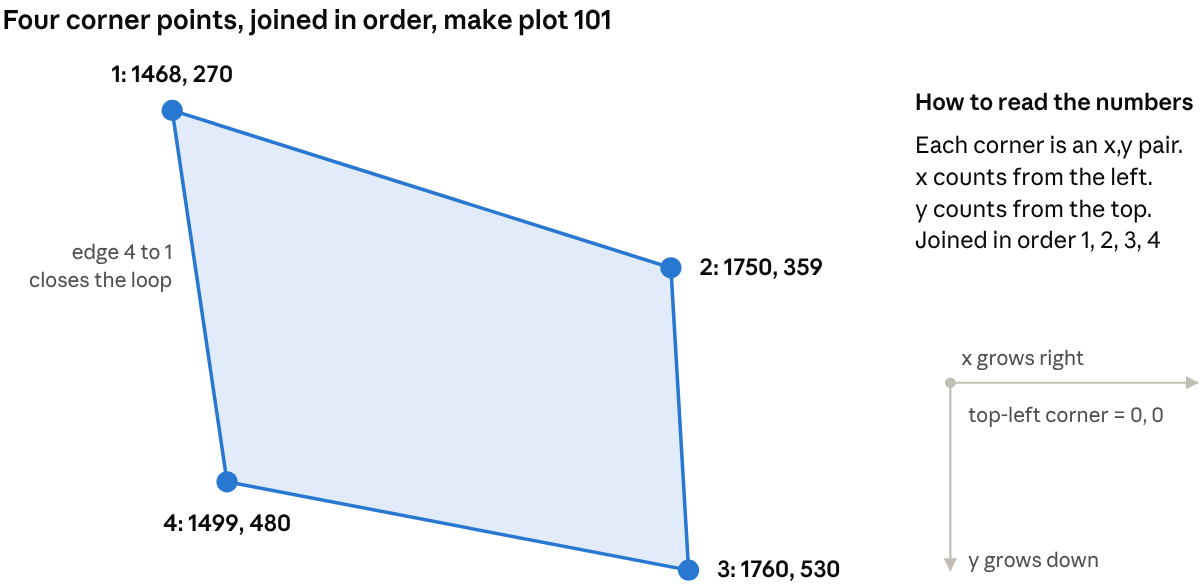


page layers · 4 layers, 1 sliding sheet

The three lower layers live on one big sheet that is wider than your screen. Move the mouse right and the sheet slides left, so you see more of the right end.

A plot is four pins joined by string

Think of each corner as a pin pushed into the map, and the plot as a string pulled round the pins in order. The drawing below is plot 101 with its real numbers.



plot 101 · 4 corner points, in order

The string runs 1 to 2 to 3 to 4, then the browser joins 4 back to 1 by itself. Each pair of numbers says how far the pin is from the left edge, then how far from the top edge.

Where the corner points are stored

Every plot is one entry in a list called `PLOTS`, inside `src/data/plots.ts`. Its corners live in one text field named `points`.

```
{
  id: '1',
  number: 101,
  status: 'AVAILABLE',
  points: '1468,270 1750,359 1760,530 1499,480',
  area: '4500 sq.ft',
  price: '$210,000',
  dimensions: { n: '80ft', s: '85ft', e: '50ft', w: '60ft' },
  center: { x: 1580, y: 440 }
}
```

Read `points` like this: each `x,y` pair is one corner, and pairs are separated by a single space. Four pairs means four corners, so this plot has four sides.

Field	What it holds	Changes the shape?
`id`	Internal name that tells plots apart	No
`number`	The plot number shown on hover (101)	No
`status`	`AVAILABLE`, `BOOKED` or `SOLD`; picks the colour and the filter	No
`points`	The corners of the polygon	Yes
`center`	Where the plot number label is placed	No
`area`, `price`, `dimensions`	Text shown in the hover cards and side panel	No

The file `src/types/index.ts` is the rulebook: it says every plot must have these fields, and `status` accepts only those three words.

Area, price, dimensions and `center` are typed in by hand. If you move a corner, none of them updates by itself.

All six plots on the board

Every corner number in the project is a position on one shared board, 3840 units wide and 1080 tall. Here are all six plots drawn from their real numbers.

Six plots sit side by side on the board and share corners

Zoomed view of the board: x from 1400 to 2400, y from 200 to 1200



board map · 6 plots, drawn from their real corner numbers

The colours show each plot's status. The little dots are the corner points; the shapes touch because neighbouring plots reuse almost the same corners.

How the code draws the polygons

The file `src/components/PlotOverlay.tsx` turns each entry of `PLOTS` into one `<polygon>` inside a single `<svg>`. SVG is the browser's built-in way to draw shapes from numbers.

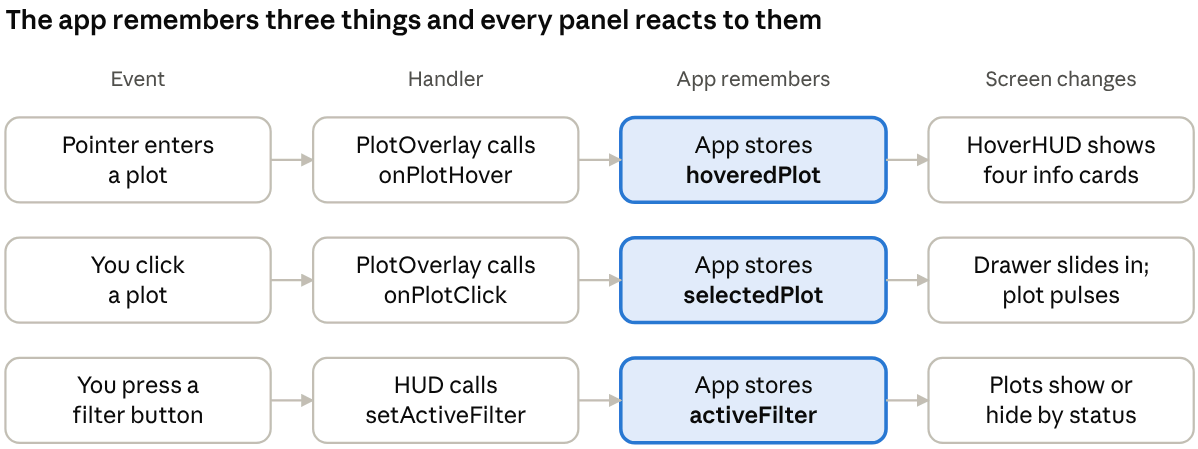
Give a polygon the string `1468,270 1750,359 1760,530 1499,480` and the browser draws a straight line from corner 1 to 2, 2 to 3, 3 to 4, then back to 1. It then fills the inside. That is the whole trick.

Code	What it does in plain words
`<svg viewBox="0 0 3840 1080">`	Opens a drawing board 3840 units wide and 1080 tall. Every corner number is measured on this board.
`PLOTS.map((plot) => ...)`	A loop. The code inside runs once per plot, so six plots give six shapes.
`isVisible = ...`	Decides whether this plot is drawn. With no status filter on, all plots show; with a status filter on, only matching plots show.
`if (!isVisible) return null`	Skips drawing plots that the filter hides.
`statusColors[plot.status]`	Picks green for AVAILABLE, yellow for BOOKED, red for SOLD. At start the filter is `NONE`, so every plot gets a white outline and a faint white fill.
`<motion.polygon points={plot.points}>`	The shape itself. The word `motion` only means it can be animated.
`onClick`, `onPointerEnter`, `onPointerLeave`	Tell the app when a plot is clicked, hovered or left.
`animate={isSelected ? ... }`	The clicked plot pulses: its outline swells from 4 to 7 and back, with a white glow. Others stay at thickness 3.
`<motion.text x={plot.center.x} y={plot.center.y}>`	Writes the plot number at the `center` point. It stays invisible until you hover.

The polygon and its number are wrapped together in one group (`<g className="group">`). That is why hovering the shape makes its number appear.

What happens when you hover or click

The plot shapes do not show anything themselves. They report what you did to `App.tsx`, which remembers it, and the other panels redraw from that memory.



event flow · 3 lanes, 3 remembered values

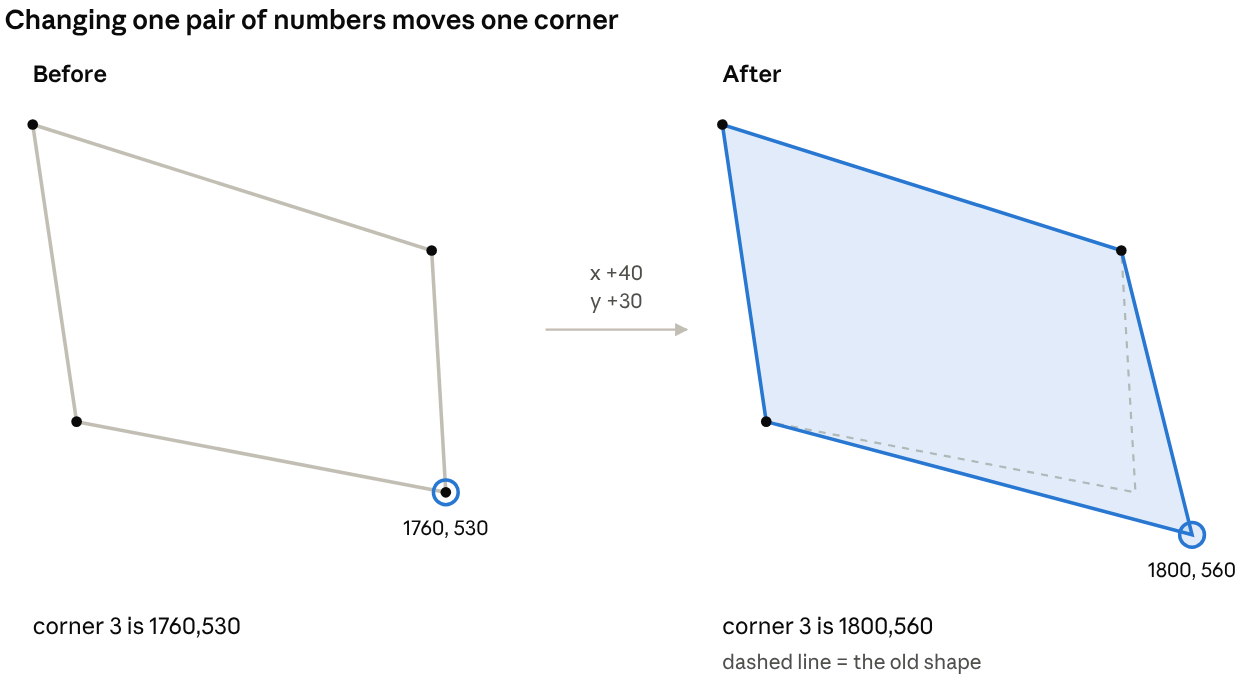
Each row is one thing you can do. The highlighted middle column is the memory kept in `App.tsx`: `hoveredPlot`, `selectedPlot` and `activeFilter`. The side drawer also redraws the clicked plot as a small shape, explained below.

How to move a point

A corner moves when you change its two numbers in `src/data/plots.ts` and save. This project has no mouse-dragging of corners; a drag-and-drop editor would need new code that listens to the mouse and rewrites these numbers.

On the board, x counts from the left edge and y counts from the top edge. A bigger x moves a corner right, and a bigger y moves it down.

1. Open `src/data/plots.ts`.
2. Find the plot by its `number`, then find its `points` line.
3. Count to the corner you want: the first pair is corner 1, the second is corner 2, and so on.
4. Change that pair. For example, `1760,530` becomes `1800,560`, which pushes corner 3 of plot 101 right by 40 and down by 30.
5. Save the file. While `npm run dev` is running, the browser refreshes by itself.
6. Fix the neighbours. Plots reuse nearly the same corner numbers, such as `1750,359` for plot 101 and `1750,360` for plot 102. Move both, or a gap or overlap appears.
7. If the shape moved far, update `center` so the number label stays inside, and retype `area` and `dimensions` by hand.



edit example · plot 101, corner 3 moved

Only corner 3 moves. The other three pairs stay as they were, so the shape stretches toward the new spot.

Finding the numbers for a new corner

The picture `public/map-bg.png` is 2172 x 724 pixels. The page stretches it to fill the 3840 x 1080 board and trims 100 units from the top and bottom.

Board number	Formula	Example for picture pixel (1000, 300)
x	picture x × 1.768	1768
y	picture y × 1.768 − 100	430

These formulas come from reading the page's CSS (`object-cover` , centred), not from testing in a browser. Check them against one corner you already know.

The small shape in the side panel

When you click a plot, `src/components/DetailsDrawer.tsx` redraws it on a tiny 100 x 100 board. The helper `getNormalizedPoints` shrinks the real numbers so any plot fits, however big or far across the map it sits.

1. Split the `points` text into pairs of numbers.
2. Find the smallest and largest x and y. Their gap is the plot's width and height.
3. Work out the scale: the smaller of `80 / width` and `80 / height`. The shape then fills at most 80 of the 100 units, which leaves a margin.
4. Shift every corner so the shape sits in the middle: `nx = (x − minX) × scale + offsetX`, and the same for y.

Worked example for plot 101, where width = 292, height = 260, scale = 0.274, offsetX = 10 and offsetY = 14.4:

Corner	Real board numbers	Small board numbers
1	1468, 270	10.0, 14.4
2	1750, 359	87.3, 38.8
3	1760, 530	90.0, 85.6
4	1499, 480	18.5, 71.9

The drawer then draws that polygon in the plot's status colour, puts a small pulsing white dot on every corner, and writes the N, S, E and W lengths around it.

Things to watch out for

- Plot 104's lower corners sit at y = 1180 and y = 1160, below the board's bottom edge at 1080, so its tip is cut off on screen.
- The list holds plots 101 to 105 and 107. There is no plot 106.
- Area, price, dimensions and `center` are typed by hand. Moving a corner never changes them.
- The "Coordinates" card on hover is a display formula built from `center` (for example `center.y / 10`). It is not a real map position.
- The corner numbers are tied to the stretched `map-bg.png`. Swap in a different picture and every corner needs re-measuring.
- `Cursor.tsx` and `EnvironmentEffects.tsx` sit in `src/components`, but `App.tsx` does not use them.

Glossary

Word	Plain meaning
Point or corner	One spot on the board, written as two numbers: `x,y`
Polygon	A closed shape made by joining points in order with straight lines
Coordinates	The pair of numbers that name a spot: how far from the left (x) and how far from the top (y)
Board	The 3840 x 1080 drawing space that all corner numbers are measured on
SVG	The browser's way of drawing shapes, lines and text from code
`viewBox`	The size of the SVG board the numbers belong to
Component	A reusable piece of the page, such as `PlotOverlay`, written as one function
State	A value the app remembers and reacts to, such as the selected plot
Filter	The status chosen on the bottom bar: all plots, status colours, Available, Booked or Sold
Hover	Resting the mouse pointer on something without clicking
HUD	The floating panels over the map: the filter bar, the hover cards and the side drawer
Pan	Sliding the big map sideways or up and down as the mouse moves

Sources

Read on 2026-10-27 from the public repository vatsalvarma/plot-mapping-try-, which is the code behind the live site.

- `src/data/plots.ts`, `src/types/index.ts`, `src/App.tsx`
- `src/components/PlotOverlay.tsx`, `MapContainer.tsx`, `DetailsDrawer.tsx`, `HoverHUD.tsx`, `HUD.tsx`
- `vite.config.ts` and `package.json`, plus the size of `public/map-bg.png`